Práctica 1: Lectura y adquisición de datos de un receptor GPS

Grupo - 7
Miembros:

- *Bruno Cabello Martín*
- *Samuel García Carretero*
- *Antonio Martín Corchón*
- *Julian Eusebio Sabroso*

1 - Análisis y Explicación del código

El código consta de 3 archivos .py necesarios para la ejecución de la aplicación.

Práctica 1 - GPS (Principal/Main)

```
import serial

from parseo_tramas import parsear_gga
from conversion_utm import geograficas_a_utm
```

Se utiliza la librería PySerial para comunicar con el receptor GPS por puerto.
Se importan las funciones principales de los otros dos scripts (`parsear_gga` y `geograficas_a_utm`)

```
def main() -> None:
    print("Práctica 1 -- Lectura GPS (NMEA GGA) → UTM (WGS84)\n")

    puerto = input("Puerto serie (ej: COM8, /dev/ttyUSB0):
").strip()
    huso = 30
```

Función Main, donde se pide al usuario el nombre del puerto (suele ser COM en Windows), ya que puede variar de un PC a otro.
Se establece el huso a 30 en España.

```
ser = None
    try:
```

```
        ser = conectar_gps(puerto)
        print(f"\n[OK] Conectado al receptor GPS en {puerto} (4800
bps, 8N1).")
        print("Leyendo tramas GGA (Ctrl+C para parar)...\n")

        contador_gga = 0
```

Se inicializa ser para cerrar el puerto, se abre el puerto y se inicializa un contador (contador_gga), para numerar las tramas GGA procesadas.

```
while True:
            linea = ser.readline().decode("ascii",
errors="replace").strip()
            if not linea:
                continue

            if "$GPGGA" not in linea and "$GNGGA" not in linea:
                continue

            contador_gga += 1
            datos = parsear_gga(linea)

            if datos is None or datos.get("calidad_fix", 0) == 0:
                print(f"[Trama {contador_gga}] Sin fix / sin
coordenadas aún.")
                continue

            utm = geograficas_a_utm(
                datos["latitud_decimal"],
                datos["longitud_decimal"],
                elipsoide="WGS84",
                huso=huso,
            )

            print(f"--- Trama GGA #{contador_gga} ---")
            print(f"Hora UTC:   {datos['hora_utc']}")
            print(f"Latitud:    {datos['latitud_decimal']:.8f}°")
            print(f"Longitud:   {datos['longitud_decimal']:.8f}°")
            print(f"Calidad:    {datos['calidad_fix']} --
{datos['calidad_texto']}")
            print(f"Satélites:  {datos['num_satelites']} | HDOP:
```

```
 {datos['hdop']}")
            print(f"Altitud:    {datos['altitud']} m")
            print(f"UTM (WGS84) -- Huso {utm['huso']}")
            print(f"  Easting (X):  {utm['easting']:.3f} m")
            print(f"  Northing (Y): {utm['northing']:.3f} m")
            print()
```

Bucle infinito para leer las tramas en tiempo real
ser.readline, para leer hasta el final tal y como llegan en NMEA.
Se filtra solo sentencias GGA : $GPGGA y $GNGGA.
Se va incrementando el contador por cada trama GGA válida y luego parsea_gga() analiza la trama y devuelve un diccionario de toda la información: hora UTC, latitud/longitud,calidad del fix, satélites, etc.

Si datos es None, todavía no engancha el receptor datos de posicionamiento (mala cobertura o al inicio de arrancar)

Cuando hay fix, se reconvierte de latitud y longitud (WGS84) a UTM mediante geográficas_a_utm, haciendo uso del elipsoide y el huso en España mostrando finálmente los resultados.

Parseo_tramas

Módulo de parseo de tramas NMEA GGA.
Analiza tramas del estándar NMEA 0183 con sentencia GGA y extrae:
hora UTC, latitud, longitud, calidad del fix, satélites, HDOP, altitud, etc.

```
CALIDAD_FIX = {
    0: "Inválido (sin fix)",
    1: "GPS fix (SPS)",
    2: "DGPS fix",
    3: "PPS fix",
    4: "Real Time Kinematic (RTK)",
    5: "Float RTK",
    6: "Estimado (dead reckoning)",
    7: "Modo de entrada manual",
    8: "Modo de simulación"
}
```

Este fragmento define un diccionario que sirve para traducir los valores numéricos de la calidad del GPS a texto entendible.

El receptor GPS devuelve un número (del 0 al 8) indicando el tipo o calidad del posicionamiento, y este diccionario permite convertir ese número en una descripción clara para el usuario.

```
def verificar_checksum(trama):
       try:
        if '$' not in trama or '*' not in trama:
            return False

        contenido = trama.split('$')[1].split('*')[0]
        checksum_recibido = trama.split('*')[1].strip()

               checksum_calculado = 0
        for char in contenido:
            checksum_calculado ^= ord(char)

        return f"{checksum_calculado:02X}" == checksum_recibido.upper()

    except (IndexError, ValueError):
        return False
```

Verifica si una trama NMEA es válida comparando el checksum recibido con uno calculado.

Extrae los caracteres entre $ y *, hace un XOR de todos ellos y lo compara con el valor de la trama.

Devuelve True si coinciden y False si hay error.

Permite asegurar que los datos del GPS no están corruptos antes de procesarlos.

```
def parsear_coordenada_latitud(valor_str, direccion):
       if not valor_str or not direccion:
        return None
    try:
        valor = float(valor_str)
        grados = int(valor / 100)
        minutos = valor - (grados * 100)
        decimal = grados + (minutos / 60.0)
        if direccion == 'S':
            decimal = -decimal
        return (grados, minutos, decimal, direccion)
    except ValueError:
        return None
```

Convierte la latitud del formato NMEA DDMM.MMMM a grados decimales.
Separa grados y minutos, convierte los minutos a fracción decimal y ajusta el signo si es Sur.
Devuelve una tupla con grados, minutos, valor decimal y dirección (N/S).
Permite usar la latitud en cálculos, mapas o conversiones UTM.

```
def parsear_coordenada_longitud(valor_str, direccion):
        if not valor_str or not direccion:
        return None
    try:
        valor = float(valor_str)
        grados = int(valor / 100)
        minutos = valor - (grados * 100)
        decimal = grados + (minutos / 60.0)
        if direccion == 'W':
            decimal = -decimal
        return (grados, minutos, decimal, direccion)
    except ValueError:
        return None
```

Convierte la longitud del formato NMEA DDDMM.MMMM a grados decimales.
Separa grados y minutos, convierte minutos a decimal y ajusta el signo si es Oeste.
Devuelve una tupla con grados, minutos, valor decimal y dirección (E/W).

Permite usar la longitud en cálculos geográficos o conversiones UTM.

```
def formatear_hora(hora_str):
        if not hora_str or len(hora_str) < 6:
        return "N/A"
    return f"{hora_str[0:2]}:{hora_str[2:4]}:{hora_str[4:]} UTC"
```

Transforma la hora del GPS (HHMMSS o HHMMSS.SS) a formato legible HH:MM:SS UTC.
Comprueba que la cadena sea válida y devuelve "N/A" si no lo es.
Permite mostrar la hora del GPS de manera comprensible para el usuario.

```
def parsear_gga(trama):
        if not trama or 'GGA' not in trama:
        return None

      if '*' in trama:
        if not verificar_checksum(trama):
            print(f"  [AVISO] Checksum inválido: {trama}")

    try:
        trama_limpia = trama.split('*')[0] if '*' in trama else trama
        campos = trama_limpia.split(',')

        if len(campos) < 15:
            return None

        hora_utc = formatear_hora(campos[1])

        lat = parsear_coordenada_latitud(campos[2], campos[3])
        if lat is None:
            return None
        lat_grados, lat_minutos, lat_decimal, lat_dir = lat


        lon = parsear_coordenada_longitud(campos[4], campos[5])
        if lon is None:
            return None
        lon_grados, lon_minutos, lon_decimal, lon_dir = lon
```

```
        calidad_fix = int(campos[6]) if campos[6] else 0
        num_satelites = int(campos[7]) if campos[7] else 0
        hdop = float(campos[8]) if campos[8] else 0.0
        altitud = float(campos[9]) if campos[9] else 0.0
        altura_geoide = float(campos[11]) if campos[11] else 0.0

        return {
            'trama_original': trama,
            'hora_utc': hora_utc,
            'latitud_grados': lat_grados,
            'latitud_minutos': lat_minutos,
            'latitud_decimal': lat_decimal,
            'latitud_direccion': lat_dir,
            'longitud_grados': lon_grados,
            'longitud_minutos': lon_minutos,
            'longitud_decimal': lon_decimal,
            'longitud_direccion': lon_dir,
            'calidad_fix': calidad_fix,
            'calidad_texto': CALIDAD_FIX.get(calidad_fix, "Desconocido"),
            'num_satelites': num_satelites,
            'hdop': hdop,
            'altitud': altitud,
            'altura_geoide': altura_geoide,
        }

    except (ValueError, IndexError) as e:
        print(f"  [ERROR] Parseando trama GGA: {e}")
        return None
```

Función principal que analiza una trama GGA completa del GPS.
Verifica checksum, separa campos, convierte coordenadas y extrae datos como hora, altitud, satélites y calidad de fix.
Devuelve un diccionario con toda la información lista para usar, mostrar o convertir a UTM.
Si la trama es inválida, devuelve None.

Conversion_utm

```python
import math

ELIPSOIDES = {
    'WGS84': {
        'nombre': 'WGS84',
        'a': 6378137.0,
        'f': 1.0 / 298.25722,
        'e2': 0.00669437999013,
        'c': 6399593.626,
    },
    'ED50': {
        'nombre': 'Europeo 1950 (ED50 / Hayford)',
        'a': 6378388.0,
        'f': 1.0 / 297.0,
        'e2': 0.00672267,
        'c': 6399936.608,
    }
}
```

Elipsodides es un diccionario que actúa de base de datos para guardar los 2 modelos: el WG84, usado por los satélites y móviles para el GPS, y el ED50, que se usaba antes de la estandarización del GPS.

```python
def geograficas_a_utm(latitud_deg, longitud_deg, elipsoide='WGS84', huso=30):
    if elipsoide.upper() not in ELIPSOIDES:
        raise ValueError(f"Elipsoide '{elipsoide}' no soportado. Use 'WGS84' o 'ED50'.")
```

Comprueba que el elipsoide es o WGS84 o ED50

```python
    elip = ELIPSOIDES[elipsoide.upper()]
    a = elip['a']
    e2 = elip['e2']
```

Extrae el semieje mayor en la variable a y la excentricidad al cuadrado en la e2 del modelo elegido.

```
phi = math.radians(latitud_deg)
lam = math.radians(longitud_deg)
```

Convertimos los grados de latitud y longitud a radianes ya que las funciones solo funcionan con radianes

```
lambda_0g = huso * 6 - 183
lambda_0 = math.radians(lambda_0g)
```

Calculamos la zona UTM en la que estamos y la pasamos a radianes

```
e_prima2 = e2 / (1.0 - e2)

N = a / math.sqrt(1.0 - e2 * math.sin(phi) ** 2)

T = math.tan(phi) ** 2

C = e_prima2 * math.cos(phi) ** 2

A = math.cos(phi) * (lam - lambda_0)



e2_2 = e2 ** 2
e2_3 = e2 ** 3

M = a * (
   (1.0 - e2 / 4.0 - 3.0 / 64.0 * e2_2 - 5.0 / 256.0 * e2_3) * phi
   - (3.0 / 8.0 * e2 + 3.0 / 32.0 * e2_2 + 45.0 / 1024.0 * e2_3)
     * math.sin(2.0 * phi)
   + (15.0 / 256.0 * e2_2 + 45.0 / 1024.0 * e2_3)
     * math.sin(4.0 * phi)
   - (35.0 / 3072.0 * e2_3)
     * math.sin(6.0 * phi)
```

```
    )
```

Variable que calcula la medición desde el ecuador hasta nuestra latitud

```
    easting = 0.9996 * N * (
        A
        + (1.0 - T + C) * A ** 3 / 6.0
        + (5.0 - 18.0 * T + T ** 2 + 72.0 * C - 58.0 * e_prima2)
          * A ** 5 / 120.0
    ) + 500000.0
```

Variable que calcula la coordenada horizontal añadiendo 500000.0 metros, conocido como Falso Este que sirve para evitar que el sistema devuelva coordenadas negativas al moverte hacia el oeste de nuestro huso. Además, aplica el factor de reducción estándar de UTM.

```
    northing = 0.9996 * (
        M + N * math.tan(phi) * (
            A ** 2 / 2.0
            + (5.0 - T + 9.0 * C + 4.0 * C ** 2) * A ** 4 / 24.0
            + (61.0 - 58.0 * T + T ** 2 + 600.0 * C - 330.0 * e_prima2)
              * A ** 6 / 720.0
        )
    )
```

Variable que calcula la coordenada vertical y también añade el factor de reducción estándar de UTM para adaptar la distancia al mapa

```
    if latitud_deg < 0:
        northing += 10000000.0
```

Si estuviéramos al sur del ecuador la coordenada Norte daría número negativo, de ahí que sumemos 10000000.0 metros para que no se trabaje con el número en negativo

```
    return {
        'easting': easting,
        'northing': northing,
        'huso': huso,
        'elipsoide': elip['nombre'],
```

```
}
```

2 - Conclusión

Este conjunto de scripts constituye una herramienta completa de adquisición, procesado y transformación de datos geoespaciales en tiempo real.

En pocas palabras: el código convierte un receptor GPS físico en una estación de datos que no solo te dice dónde estás en el mapa, sino que traduce esas coordenadas angulares (latitud/longitud) a un sistema métrico (UTM).